Preparing Property Data for Price Prediction: A Comprehensive Data Cleaning and Feature Engineering Approach

Alya Karim

2024-11-30

Contents

1 Executive Summary

This report outlines the steps taken to prepare two property datasets—PropertyDataset1.csv and PropertyDataset2.csv—for modeling analysis to predict property prices in Ireland. The data preparation process includes identifying and removing outliers, cleaning the datasets by addressing missing values and standardizing formats, and creating additional explanatory variables to enhance the analysis. Data visualizations are also provided to highlight key trends and relationships between variables. The final cleaned and enriched dataset will allow Sarah to proceed directly to modeling, with a comprehensive and well-structured dataset ready for analysis.

2 Introduction

This report describes the comprehensive preparation of two property-related datasets, PropertyDataset1 and PropertyDataset2, for analysis and predictive modeling. These datasets contain critical information on property transactions in Ireland, such as sale prices, transaction dates, property addresses, and descriptive details. The primary goal is to transform these datasets into a unified, clean, and structured format suitable for advanced analysis, including exploratory data analysis (EDA) and predictive modeling.

To begin, the analysis environment is configured by setting the CRAN mirror to ensure seamless access to up-to-date R packages. The tinytex package is installed to enable PDF generation for reporting purposes. Subsequently, essential libraries are loaded to support various stages of the data preparation process. These libraries include dplyr for data manipulation tasks such as renaming columns and combining datasets, ggplot2 for creating visualizations to explore relationships and trends, and tidyr for reshaping and organizing data into a tidy format. Additionally, lubridate is used to handle and standardize date fields, while stringr and stringi are employed to clean and manipulate text fields, such as addresses and property descriptions. Other libraries, such as tidyselect, assist in programmatically renaming and selecting columns.

The datasets are loaded into R using the read.csv() function. For PropertyDataset2, the file encoding is explicitly set to ISO-8859-1 to address special character issues commonly encountered in international datasets. An initial inspection of both datasets is performed using the head() function to understand their structure, assess the quality of the data, and identify any inconsistencies or missing information.

To ensure consistency between the two datasets, column names are standardized. This step is critical for seamless integration during the merging process. For PropertyDataset1, the column names are updated to include additional fields, such as geohash and raw_address, which provide enhanced location-specific information. Similarly, the column names in PropertyDataset2 are aligned with the schema of PropertyDataset1 wherever applicable. This harmonization eliminates discrepancies that could otherwise introduce errors during data manipulation.

The outcome of this data preparation process is a high-quality, unified dataset that integrates information from both sources. This dataset is now ready for detailed exploratory data analysis, visualization, and predictive modeling. The effort invested in standardizing, cleaning, and enriching the data ensures that subsequent analyses are accurate and insightful, providing a solid foundation for understanding trends in property transactions and developing predictive models to estimate property prices in Ireland.

```
#Set CRAN mirror
options(repos = c(CRAN = "https://cran.rstudio.com/"))

tinytex::install_tinytex(force = TRUE)

#Load necessary libraries
library(dplyr)
library(ggplot2)
library(tidyr)
```

```
library(tidyselect)
library(lubridate)
library(stringr)
library(tinytex)
library(stringi)

#Load the datasets
propertydataset1 <- read.csv("PropertyDataset1.csv")
propertydataset2 <- read.csv("PropertyDataset2.csv", fileEncoding = "ISO-8859-1")

cleanpropertydataset1 <- propertydataset1
cleanpropertydataset2 <- propertydataset2

#Inspect the first few rows of both datasets
head(cleanpropertydataset1)
```

```
##        geohash  sale_date     price
## 1              29/05/2020  63436.12
## 2 gce8t0tbed4u 17/12/2019 341000.00
## 3 gc6g7r445n7p 09/02/2017 170000.00
## 4 gc7j3n5rvwmc 19/12/2018 112000.00
## 5 gc65bu202d1y 31/03/2017 173000.00
## 6 gc7rb1tmkzz8 08/03/2019 405000.00
##                                 formatted_address
## 1
## 2                 4 Woodland Park, Rush, Co. Dublin
## 3 50 Marble Crest, Kilkenny, Kilkenny, Co. Kilkenny
## 4       18 Marina Court, Athy, Kildare, Co. Kildare
## 5               61 Rosehill, Newport, Co. Tipperary
## 6
##                                     raw_address post_code    county
## 1 APARTMENT NO. 7, LIBRARY CORNER, MANORHAMILTON             Leitrim
## 2                  4 WOODLAND PARK, RUSH, DUBLIN              Dublin
## 3            50 MARBLE CREST, KILKENNY, KILKENNY            Kilkenny
## 4                 18 MARINA COURT, ATHY, KILDARE             Kildare
## 5             61 ROSEHILL, NEWPORT, CO TIPPERARY           Tipperary
## 6        22 Fenton Green, Church Street, Kilcock             Kildare
##   num_bedrooms num_bathrooms                  property_description
## 1                                  New Dwelling house /Apartment
## 2   3 Bedrooms   2 Bathrooms Second-Hand Dwelling house /Apartment
## 3                            Second-Hand Dwelling house /Apartment
## 4                            Second-Hand Dwelling house /Apartment
## 5   4 Bedrooms   3 Bathrooms Second-Hand Dwelling house /Apartment
## 6                                  New Dwelling house /Apartment
##   property_size_description                formatted_description
## 1
## 2                                            Semi-Detached House
## 3                          Second-Hand Dwelling House/Apartment
## 4                          Second-Hand Dwelling House/Apartment
## 5                                            Semi-Detached House
## 6
##   not_full_market_price vat_exclusive parking parking1
## 1                 FALSE          TRUE     Has  Parking
```

```
## 2                FALSE         FALSE     Has  Parking
## 3                FALSE         FALSE     Has  Parking
## 4                FALSE         FALSE     Has  Parking
## 5                FALSE         FALSE     Has  Parking
## 6                FALSE          TRUE     Has  Parking
```

```
head(cleanpropertydataset2)
```

```
##   Date.of.Sale..dd.mm.yyyy.                                      Address
## 1                01/01/2010         5 Braemor Drive, Churchtown, Co.Dublin
## 2                03/01/2010 134 Ashewood Walk, Summerhill Lane, Portlaoise
## 3                04/01/2010            1 Meadow Avenue, Dundrum, Dublin 14
## 4                04/01/2010                        1 The Haven, Mornington
## 5                04/01/2010                  11 Melville Heights, Kilkenny
## 6                04/01/2010                 12 Sallymount Avenue, Ranelagh
##     County        Price.... Not.Full.Market.Price VAT.Exclusive
## 1   Dublin \u0080343,000.00                    No            No
## 2    Laois \u0080185,000.00                    No           Yes
## 3   Dublin \u0080438,500.00                    No            No
## 4    Meath \u0080400,000.00                    No            No
## 5 Kilkenny \u0080160,000.00                    No            No
## 6   Dublin \u0080425,000.00                    No            No
##                 Description.of.Property
## 1 Second-Hand Dwelling house /Apartment
## 2         New Dwelling house /Apartment
## 3 Second-Hand Dwelling house /Apartment
## 4 Second-Hand Dwelling house /Apartment
## 5 Second-Hand Dwelling house /Apartment
## 6 Second-Hand Dwelling house /Apartment
##                                         Property.Size.Description
## 1
## 2 greater than or equal to 38 sq metres and less than 125 sq metres
## 3
## 4
## 5
## 6
```

```
#Standardize column names for consistency
colnames(cleanpropertydataset1) <- c("geohash","sale_date", "price", "formatted_address",
                                     "raw_address", "post_code", "county", "num_bedrooms",
                                     "num_bathrooms", "property_description",
                                     "property_size_description", "formatted_description",
                                     "not_full_market_price", "vat_exclusive",
                                     "parking", "parking1")

colnames(cleanpropertydataset2) <- c("sale_date", "formatted_address", "county", "price",
                          "not_full_market_price", "vat_exclusive",
                          "property_description", "property_size_description")
```

3 Data Cleaning

3.1 Data Cleaning: Removing Duplicates and Checking Data Types

Data cleaning begins with inspecting the sale_date column in PropertyDataset2 to understand its structure and identify any anomalies. Displaying the first few rows using the head() function helps confirm that the data in this column is formatted correctly and consistent across entries.

The data types of all variables in PropertyDataset1 and PropertyDataset2 are then checked using the sapply() function. Ensuring that each column has the correct data type is essential for performing accurate calculations and analyses. For example, columns such as price should be numeric, and sale_date should be in date format.

Next, duplicate rows are removed from both datasets using the distinct() function from the dplyr package. Eliminating duplicates ensures that each entry is unique, preventing redundancy and improving data accuracy. This step is critical for maintaining data integrity and ensuring reliable results during subsequent analysis.

```
#Check the first few rows of data
invisible(head(cleanpropertydataset1))
invisible(head(cleanpropertydataset2))

#Check data type
sapply(cleanpropertydataset1,class)
```

```
##                   geohash                 sale_date                     price
##               "character"               "character"                 "numeric"
##         formatted_address              raw_address               post_code
##               "character"               "character"               "character"
##                    county             num_bedrooms            num_bathrooms
##               "character"               "character"               "character"
##      property_description property_size_description     formatted_description
##               "character"               "character"               "character"
##     not_full_market_price             vat_exclusive                   parking
##               "character"               "character"               "character"
##                  parking1
##               "character"
```

```
sapply(cleanpropertydataset2,class)
```

```
##                 sale_date         formatted_address                    county
##               "character"               "character"               "character"
##                      price     not_full_market_price             vat_exclusive
##               "character"               "character"               "character"
##      property_description property_size_description
##               "character"               "character"
```

```
#Remove duplicate rows from the dataset
cleanpropertydataset1 <- distinct(cleanpropertydataset1)

#Remove duplicate rows 2 from the dataset
cleanpropertydataset2 <- distinct(cleanpropertydataset2)
```

3.2 Date Formatting and Cleaning

In this section, we focus on cleaning and formatting the sale_date column in both propertydataset1 and propertydataset2. First, we check the data type of the sale_date column in both datasets to ensure that it is properly recognized as a date variable. We then handle any empty strings or unexpected values by replacing them with NA using the na_if() function. This ensures that any missing or incorrectly formatted date entries are correctly flagged as missing values, preventing issues in analysis.

Next, we format the sale_date column in both datasets using the dmy() function from the lubridate package, which converts the dates to the day-month-year format. This standardizes the date format and ensures consistency across the datasets. Finally, we check for any unusual characters or formatting issues in the sale_date column of propertydataset2 by reviewing the unique values in the column. This step helps us identify any remaining discrepancies in the date entries that may need further cleaning.

```
##Date

#Check the first few rows of the sale_date column
head(cleanpropertydataset1$sale_date)
```

```
## [1] "29/05/2020" "17/12/2019" "09/02/2017" "19/12/2018" "31/03/2017"
## [6] "08/03/2019"
```

```
head(cleanpropertydataset2$sale_date)
```

```
## [1] "01/01/2010" "03/01/2010" "04/01/2010" "04/01/2010" "04/01/2010"
## [6] "04/01/2010"
```

```
#Check data type sale_date
class(cleanpropertydataset1$sale_date)
```

```
## [1] "character"
```

```
class(cleanpropertydataset2$sale_date)
```

```
## [1] "character"
```

```
#Replace empty strings or unexpected values with NA
cleanpropertydataset1$sale_date <- na_if(cleanpropertydataset1$sale_date, "")

#Format sale_date in PropertyDataset1
cleanpropertydataset1$sale_date <- dmy(cleanpropertydataset1$sale_date)

#Replace empty strings or unexpected values with NA
cleanpropertydataset2$sale_date <- na_if(cleanpropertydataset2$sale_date, "")

#Format sale_date in PropertyDataset2
cleanpropertydataset2$sale_date <- dmy(cleanpropertydataset2$sale_date)

#Check for unusual characters or formatting issues
invisible(unique(cleanpropertydataset2$sale_date))
```

3.3 Address Cleaning and Formatting

This section focuses on ensuring the sale_date column in both datasets is clean, correctly formatted, and standardized. The first step involves checking the data type of the sale_date column in both datasets to confirm that it is properly recognized as a date variable. This validation is crucial for performing any date-based calculations or analyses accurately.

Empty strings or unexpected values in the sale_date column are replaced with NA using the na_if() function. This step ensures that missing or incorrectly formatted date entries are properly flagged as missing values, avoiding potential issues during analysis.

The sale_date column is then formatted using the dmy() function from the lubridate package, which converts the dates into a day-month-year format. This standardization ensures consistency in date representation across both datasets.

Finally, the unique() function is used to review all distinct values in the sale_date column of Property-Dataset2, helping to identify any unusual characters or formatting discrepancies that may require additional cleaning.

```
##Address

#Delete the 'formatted_address' column and rename 'raw_address' to 'formatted_address'
cleanpropertydataset1 <- cleanpropertydataset1 %>%
  select(-formatted_address) %>%  # Remove the 'formatted_address' column
  rename(formatted_address = raw_address)  # Rename 'raw_address' to 'formatted_address'

#For propertydataset1 change to Sentence Case
cleanpropertydataset1$formatted_address <-
  str_to_title(cleanpropertydataset1$formatted_address)

#For propertydataset2 change to Sentence Case
cleanpropertydataset2$formatted_address <-
  str_to_title(cleanpropertydataset2$formatted_address)

#dataset1
#Remove HTML entities (e.g., &#039;) and special characters with actual single quote
cleanpropertydataset1$formatted_address <-
  gsub("&#039;", "'", cleanpropertydataset1$formatted_address)
# Remove non-alphanumeric characters (except commas, periods, and spaces)
cleanpropertydataset1$formatted_address <-
  gsub("[^[:alnum:][:space:],.]", "", cleanpropertydataset1$formatted_address)

#dataset2
#Remove HTML entities (e.g., &#039;) and special characters
cleanpropertydataset2$formatted_address <-
  gsub("&#039;", "'", cleanpropertydataset2$formatted_address)
cleanpropertydataset2$formatted_address <-
  gsub("[^[:alnum:][:space:],.]", "", cleanpropertydataset2$formatted_address)

#Extract Dublin followed by a number (e.g., Dublin 11, Dublin 2)
cleanpropertydataset1$post_code <-
  str_extract(cleanpropertydataset1$formatted_address, "Dublin \\d+")
cleanpropertydataset2$post_code <-
  str_extract(cleanpropertydataset2$formatted_address, "Dublin \\d+")
```

3.4 Property Description and Property Size Description Cleaning

This section focuses on cleaning and standardizing two key columns: property_description and property_size_description. By addressing inconsistencies and extracting meaningful information, the data becomes more structured and ready for analysis.

3.4.1 Property Description Standardization

The property_description column often contains variations in phrasing, capitalization, and formatting. For example, the term "second-hand dwelling house/apartment" can appear as:

- second-hand dwelling house / apartment
- Second-Hand Dwelling house /Apartment
- Teach/Árasán Cónaithe Atháimhe (Irish translation)

To standardize these variations:

- The gsub() function is used to search for all known variations and replace them with a consistent form: "second-hand dwelling house/apartment".
- A similar approach is applied to standardize variations of "new dwelling house/apartment".
- The ignore.case = TRUE argument ensures matches are case-insensitive. = For additional formatting, the descriptions are converted to sentence case using str_to_title() from the stringr package.

Finally, if a formatted_description column exists and contains non-empty values, it replaces the property_description column for better accuracy. The formatted_description column is then removed to avoid redundancy.

3.4.2 Property Size Description Cleaning

The property_size_description column often contains text and numeric values, such as "100 sqm" or "2,500 sq ft". To enable quantitative analysis of property sizes:

- The numeric portion of the size is extracted using the str_extract() function from the stringr package, which identifies the first sequence of digits in the text
- A new column, property_size_numeric, is created to store these numeric values, ensuring that property sizes can be analyzed directly for further analysis, such as calculating average sizes, performing clustering, or identifying size-based outliers.

Additionally, a custom function, clean_property_size_desc(), is used to standardize and clean the property_size_description column. This function handles variations in the text, removes any unwanted characters or corrupted data, and standardizes the descriptions into consistent categories, such as "greater than or equal to 125 sq metres" and "less than 38 sq metres". This ensures that the dataset is uniform and ready for analysis.

```
##Property Description

#Standardize all variations of "second-hand dwelling house/apartment" for dataset1
cleanpropertydataset1$property_description <- gsub(
  pattern = "second-hand dwelling house / apartment|
  second-hand dwelling house /Apartment|Second-Hand Dwelling house /Apartment|
  second-hand dwelling house /Apartment|Teach/Árasán Cónaithe Atháimhe",
```

```
  replacement = "second-hand dwelling house/apartment",
  x = cleanpropertydataset1$property_description,
  ignore.case = TRUE
)

#Standardize all variations of "second-hand dwelling house/apartment" for dataset2
cleanpropertydataset2$property_description <- gsub(
  pattern = "second-hand dwelling house / apartment|
  second-hand dwelling house /Apartment|Second-Hand Dwelling house /Apartment|
  second-hand dwelling house /Apartment|Teach/Árasán Cónaithe Atháimhe",
  replacement = "second-hand dwelling house/apartment",
  x = cleanpropertydataset2$property_description,
  ignore.case = TRUE
)

#Standardize all variations of to "new dwelling house/apartment" for dataset1
cleanpropertydataset1$property_description <- gsub(
  pattern = "New Dwelling house /Apartment|Teach/Árasán Cónaithe Nua|
  Teach/\\?ras\\?n C\\?naithe Nua|new Dwelling house /Apartment|
  new dwelling house /apartment|New dwelling house /apartment|New dwelling house /Apartment",
  replacement = "new dwelling house/apartment",
  x = cleanpropertydataset1$property_description,
  ignore.case = TRUE
)

#Standardize all variations of "new dwelling house/apartment" for dataset2
cleanpropertydataset2$property_description <- gsub(
  pattern = "New Dwelling house /Apartment|Teach/Árasán Cónaithe Nua|
  Teach/\\?ras\\?n C\\?naithe Nua|new Dwelling house /Apartment|
  new dwelling house /apartment|New dwelling house /apartment|New dwelling house /Apartment",
  replacement = "new dwelling house/apartment",
  x = cleanpropertydataset2$property_description,
  ignore.case = TRUE
)

#Check the unique property descriptions again to ensure the replacements are done
unique(cleanpropertydataset1$property_description)
```

```
## [1] "new dwelling house/apartment"
## [2] "second-hand dwelling house/apartment"
```

```
unique(cleanpropertydataset2$property_description)
```

```
## [1] "second-hand dwelling house/apartment"
## [2] "new dwelling house/apartment"
## [3] "Teach/?ras?n C?naithe Nua"
```

```
#Convert 'property_description' to sentence case (capitalizing first letter of each word)
cleanpropertydataset1$property_description <-
  str_to_title(cleanpropertydataset1$property_description)

cleanpropertydataset2$property_description <-
```

```
  str_to_title(cleanpropertydataset2$property_description)

#Replace property_description with formatted_description if formatted_description exists
cleanpropertydataset1$property_description <- ifelse(
  !is.na(cleanpropertydataset1$formatted_description) &
    cleanpropertydataset1$formatted_description != "",
  cleanpropertydataset1$formatted_description,
  cleanpropertydataset1$property_description
)
#Delete the 'formatted_description' column
cleanpropertydataset1 <- cleanpropertydataset1 %>%
  select(-formatted_description)

#Verify the changes
head(cleanpropertydataset1$formatted_address)
```

```
## [1] "Apartment No. 7, Library Corner, Manorhamilton"
## [2] "4 Woodland Park, Rush, Dublin"
## [3] "50 Marble Crest, Kilkenny, Kilkenny"
## [4] "18 Marina Court, Athy, Kildare"
## [5] "61 Rosehill, Newport, Co Tipperary"
## [6] "22 Fenton Green, Church Street, Kilcock"
```

```
head(cleanpropertydataset2$formatted_address)
```

```
## [1] "5 Braemor Drive, Churchtown, Co.dublin"
## [2] "134 Ashewood Walk, Summerhill Lane, Portlaoise"
## [3] "1 Meadow Avenue, Dundrum, Dublin 14"
## [4] "1 The Haven, Mornington"
## [5] "11 Melville Heights, Kilkenny"
## [6] "12 Sallymount Avenue, Ranelagh"
```

```
##Property Size Description

unique(cleanpropertydataset1$property_size_description)
```

```
## [1] ""
## [2] "greater than 125 sq metres"
## [3] "greater than or equal to 38 sq metres and less than 125 sq metres"
## [4] "greater than or equal to 125 sq metres"
## [5] "less than 38 sq metres"
## [6] "Greater than or equal to 38 sq metres and less than 125 sq metres"
## [7] "Greater than or equal to 125 sq metres"
```

```
unique(cleanpropertydataset2$property_size_description)
```

```
## [1] ""
## [2] "greater than or equal to 38 sq metres and less than 125 sq metres"
## [3] "greater than 125 sq metres"
## [4] "less than 38 sq metres"
## [5] "greater than or equal to 125 sq metres"
```

```
## [6] "níos mó ná nó cothrom le 38 méadar cearnach agus níos lú ná 125 méadar cearnach"
## [7] "n?os l? n? 38 m?adar cearnach"
## [8] "Greater than or equal to 38 sq metres and less than 125 sq metres"
## [9] "greater than or equal to 38 sq metres and less than 125 square metres"
```

```
#Extract numeric property size from the description and create a new numeric column
cleanpropertydataset1 <- cleanpropertydataset1 %>%
  mutate(
    property_size_numeric = as.numeric(str_extract(property_size_description, "\\d+"))
  )

#Extract numeric property size from the description and create a new numeric column
cleanpropertydataset2 <- cleanpropertydataset2 %>%
  mutate(
    property_size_numeric = as.numeric(str_extract(property_size_description, "\\d+"))
  )

#Function to standardize property size descriptions without using stringi
clean_property_size_desc <- function(description) {
  #Convert to lowercase
  description <- tolower(description)
  #Remove leading and trailing spaces
  description <- trimws(description)

  #Manually replace the corrupted characters with the correct text
  description <-
    gsub("n\\?os l\\? n\\? 38 m\\?adar cearnach", "less than 38 sq metres", description)
  description <-
    gsub("n\\?os l\\? n\\? 38 sq metres", "less than 38 sq metres", description)

  #Replace variations with standardized values
  description <-
    gsub("greater than 125 sq metres", "greater than or equal to 125 sq metres", description)
  description <-
    gsub("greater than or equal to 38 sq metres and less than 125 square metres",
         "greater than or equal to 38 sq metres and less than 125 sq metres", description)
  description <-
    gsub("greater than or equal to 38 sq metres and less than 125 sq metres",
         "greater than or equal to 38 sq metres and less than 125 sq metres", description)

  #Handle non-English or corrupted text manually
  description <-
    gsub("níos mó ná nó cothrom le 38 méadar cearnach agus níos lú ná 125 méadar cearnach",
         "greater than or equal to 38 sq metres and less than 125 sq metres", description)

  #Replace empty strings with NA for consistency
  description[description == ""] <- NA

  return(description)
}

#Apply the cleaning function to both datasets' 'property_size_description' column
cleanpropertydataset1$property_size_description <-
  clean_property_size_desc(cleanpropertydataset1$property_size_description)
```

```r
cleanpropertydataset2$property_size_description <-
  clean_property_size_desc(cleanpropertydataset2$property_size_description)

#Verify cleaned unique values
unique(cleanpropertydataset1$property_size_description)
```

```
## [1] NA
## [2] "greater than or equal to 125 sq metres"
## [3] "greater than or equal to 38 sq metres and less than 125 sq metres"
## [4] "less than 38 sq metres"
```

```r
unique(cleanpropertydataset2$property_size_description)
```

```
## [1] NA
## [2] "greater than or equal to 38 sq metres and less than 125 sq metres"
## [3] "greater than or equal to 125 sq metres"
## [4] "less than 38 sq metres"
```

3.5 Price Cleaning and Conversion

This section focuses on cleaning and converting the price column in propertydataset2 to ensure that the values are properly formatted and ready for analysis.

Inspect Unique Values The first step is to inspect the unique values in the price column using the unique() function. This helps identify any inconsistencies or non-numeric characters that might be present in the data, such as currency symbols or commas, which can interfere with subsequent analysis.

Remove Non-Numeric Characters In the price column, non-numeric characters like the euro sign (ŏ080), commas, or other symbols can distort the data. These characters need to be removed to work with pure numeric values. The gsub() function is used with the regular expression [ˆ0-9.], which targets any character that is not a digit or a decimal point and replaces it with an empty string. This ensures that only numeric values remain in the price column.

Convert to Numeric Once the non-numeric characters are removed, the price column is converted to a numeric format using the as.numeric() function. This ensures that the price column contains only numeric values, which can then be used in statistical analysis or machine learning models.

These steps ensure that the price data is cleaned and formatted correctly for analysis.

```r
##Price

#Inspect unique values in the price column
invisible(unique(cleanpropertydataset1$price))
invisible(unique(cleanpropertydataset2$price))

#Remove non-numeric characters for cleanpropertydataset2
#Dataset2 because it has unique characters including \u0080 and commas
cleanpropertydataset2$price <-
  gsub("[^0-9.]", "", cleanpropertydataset2$price)

#Convert to numeric
cleanpropertydataset2$price <- as.numeric(cleanpropertydataset2$price)
```

3.6 Combining the Datasets

The code combines two datasets, cleanpropertydataset1 and cleanpropertydataset2, into a single dataset, all_properties_data. It uses the bind_rows() function from the dplyr package, which stacks the datasets vertically, meaning the rows from cleanpropertydataset2 will be appended below the rows of cleanpropertydataset1. This operation assumes that both datasets have the same structure (i.e., the same columns).

```
##Combine datasets
all_properties_data <- bind_rows(cleanpropertydataset1, cleanpropertydataset2)


#View structure and summary
str(all_properties_data) #for data types
```

```
## 'data.frame':    667131 obs. of  15 variables:
##  $ geohash                 : chr  "" "gce8t0tbed4u" "gc6g7r445n7p" "gc7j3n5rvwmc" ...
##  $ sale_date               : Date, format: "2020-05-29" "2019-12-17" ...
##  $ price                   : num  63436 341000 170000 112000 173000 ...
##  $ formatted_address       : chr  "Apartment No. 7, Library Corner, Manorhamilton" "4 Woodland Park
##  $ post_code               : chr  NA NA NA NA ...
##  $ county                  : chr  "Leitrim" "Dublin" "Kilkenny" "Kildare" ...
##  $ num_bedrooms            : chr  "" "3 Bedrooms" "" "" ...
##  $ num_bathrooms           : chr  "" "2 Bathrooms" "" "" ...
##  $ property_description    : chr  "New Dwelling House/Apartment" "Semi-Detached House" "Second-Hand
##  $ property_size_description: chr  NA NA NA NA ...
##  $ not_full_market_price   : chr  "FALSE" "FALSE" "FALSE" "FALSE" ...
##  $ vat_exclusive           : chr  "TRUE" "FALSE" "FALSE" "FALSE" ...
##  $ parking                 : chr  "Has" "Has" "Has" "Has" ...
##  $ parking1                : chr  "Parking" "Parking" "Parking" "Parking" ...
##  $ property_size_numeric   : num  NA NA NA NA NA NA NA NA NA NA ...
```

```
summary(all_properties_data) #for summary statistics
```

```
##    geohash            sale_date              price           formatted_address
##  Length:667131      Min.   :2010-01-01   Min.   :        0   Length:667131
##  Class :character   1st Qu.:2015-04-29   1st Qu.:   129750   Class :character
##  Mode  :character   Median :2018-04-11   Median :   216000   Mode  :character
##                     Mean   :2018-01-10   Mean   :   284554
##                     3rd Qu.:2021-01-27   3rd Qu.:   325991
##                     Max.   :2033-10-27   Max.   :225000000
##                     NA's   :3
##   post_code            county          num_bedrooms       num_bathrooms
##  Length:667131      Length:667131      Length:667131      Length:667131
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##
##
##
##
##  property_description property_size_description not_full_market_price
##  Length:667131        Length:667131             Length:667131
##  Class :character     Class :character          Class :character
##  Mode  :character     Mode  :character          Mode  :character
```

```
##
##
##
##
##  vat_exclusive        parking            parking1         property_size_numeric
##  Length:667131      Length:667131      Length:667131      Min.   : 38.0
##  Class :character   Class :character   Class :character   1st Qu.: 38.0
##  Mode  :character   Mode  :character   Mode  :character   Median : 38.0
##                                                           Mean   : 57.1
##                                                           3rd Qu.: 38.0
##                                                           Max.   :125.0
##                                                           NA's   :609835
```

```
colnames(all_properties_data) #for column names in dataset
```

```
##  [1] "geohash"                   "sale_date"
##  [3] "price"                     "formatted_address"
##  [5] "post_code"                 "county"
##  [7] "num_bedrooms"              "num_bathrooms"
##  [9] "property_description"      "property_size_description"
## [11] "not_full_market_price"     "vat_exclusive"
## [13] "parking"                   "parking1"
## [15] "property_size_numeric"
```

```
#To check what kind of properties that exists in the combined dataset
unique(all_properties_data$property_description)
```

```
##  [1] "New Dwelling House/Apartment"
##  [2] "Semi-Detached House"
##  [3] "Second-Hand Dwelling House/Apartment"
##  [4] "Terraced House"
##  [5] "Apartment For Sale"
##  [6] "Detached House"
##  [7] "End of Terrace House"
##  [8] "House For Sale"
##  [9] "Bungalow For Sale"
## [10] "Duplex For Sale"
## [11] "Townhouse"
## [12] "Site For Sale"
## [13] "second-hand dwelling house/apartment"
## [14] "end of terrace house"
## [15] "semi-detached house"
## [16] "Teach/?Ras?N C?Naithe Nua"
```

3.7 Identifying and Removing Outliers

Outliers are data points that significantly deviate from the rest of the dataset. They can distort statistical analysis and affect the accuracy of machine learning models. Identifying and handling outliers is a crucial step in data cleaning. This section discusses how to identify and remove outliers in the price and property_size_numeric columns, as they are critical to understanding property prices.

Identifying Outliers in the price Column

The Interquartile Range (IQR) method is used to identify outliers in the price column. The IQR is the range between the 25th percentile (Q1) and the 75th percentile (Q3). Any value below Q1 - 1.5 * IQR or above Q3 + 1.5 * IQR is considered an outlier.

```
##Quartile
#Check structure and ensure numeric columns are correct
all_properties_data$price <- as.numeric(all_properties_data$price)
all_properties_data$property_size_numeric <-
  as.numeric(all_properties_data$property_size_numeric)

#Calculate IQR for price column
Q1 <- quantile(all_properties_data$price, 0.25, na.rm = TRUE)
Q3 <- quantile(all_properties_data$price, 0.75, na.rm = TRUE)
IQR_price <- Q3 - Q1

#Identify outliers
lower_bound <- Q1 - 1.5 * IQR_price
upper_bound <- Q3 + 1.5 * IQR_price

#Remove outliers in the price column
all_properties_data <- all_properties_data %>%
  filter(!is.na(price) & price >= lower_bound & price <= upper_bound)

#Check the results
summary(all_properties_data$price)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##       0  125000  205000  224470  303965  620300
```

3.8 Reasoning for Removal of Outliers

Outliers can distort statistical analysis and affect the performance of machine learning models, particularly in key variables like price and property_size_numeric. These extreme values can skew results, making it challenging to identify meaningful patterns. Removing outliers ensures the data reflects more typical values, leading to more reliable and interpretable analysis. However, it is crucial to strike a balance between cleaning the data and preserving enough information for meaningful insights, ensuring that a substantial portion of the data remains available for analysis.

In property price analysis, addressing outliers enhances data quality and provides a clearer picture of market behavior. By removing or analyzing these values separately, patterns and trends better represent the majority of transactions, preventing them from being overshadowed by extreme cases. Additionally, outliers can offer unique insights, such as rare luxury sales or heavily discounted properties, which may hold strategic importance. This dual approach ensures both improved model performance and valuable understanding of the data's nuances.

Below is an overview of the data cleaning process applied to the property datasets:

Date Formatting: The date format was standardized in both datasets using the dmy() function from the lubridate package. This ensures that all dates are consistently formatted, which is essential for time-based analyses and predictions.

Address Cleaning: The raw_address column was renamed to formatted_address to standardize the naming convention. The addresses were then converted to sentence case using str_to_title() to ensure consistency. Additionally, any unwanted HTML entities or special characters were removed using the gsub() function. This step makes the address data more uniform and suitable for potential geospatial analysis.

Price Column Cleaning: Non-numeric characters such as the euro sign and commas were removed from the price column, and the values were converted into a numeric format. This allows for accurate numerical modeling and analysis of property prices.

Property Size Standardization: Numeric values were extracted from the property_size_description column, and a new column called property_size_numeric was created. This transformation enables property size to be treated as a numeric variable in analyses, such as regression modeling.

Outlier Removal: The Interquartile Range (IQR) method was used to identify and remove outliers from the price and property_size_numeric columns. This step ensures that extreme values do not skew the analysis and that the models focus on typical data points.

4 Creating Additional Columns for Analysis

To further enhance the dataset and provide more explanatory variables for modeling, several new columns were created to capture important aspects related to time and location. These new columns will help in uncovering trends, geographic patterns, and assist in spatial analyses.

4.1 Year of Sale

Column: sale_date

The year of sale was extracted from the sale_date column using the year() function. By capturing the year, the dataset can be analyzed to explore trends over time, such as how property prices may change from one year to another or how market conditions fluctuate annually. This additional column can be used in time-series analysis or to identify seasonal trends in the dataset.

Purpose:

Including the year of sale in the dataset allows for better analysis of temporal trends in the property market. It helps understand how property prices evolve over time, whether they are generally increasing, decreasing, or remaining stable. The year of sale is useful for analyzing market dynamics and predicting future price changes based on historical data.

Method:

- **Year Extraction:** The sale_date column was used to extract the year of sale using the year() function. This allows analysis of how property prices or market conditions fluctuate from one year to another.
- **Month Extraction:** The month of the sale was extracted as a name using the month() function with the label = TRUE argument, providing a more readable format of the month (e.g., "Jan", "Feb", etc.).
- **New Columns:** The sale_year column was added to capture the year of sale, and the sale_month_name column was added to capture the month name of the sale.

```
##YearMonth
#Extract the year
all_properties_data$sale_year <- year(all_properties_data$sale_date)

#Extract the month as a name
all_properties_data$sale_month_name <- month(all_properties_data$sale_date, label = TRUE)

#Rearrange the column
all_properties_data <- all_properties_data %>%
  select(geohash,sale_date, sale_month_name,sale_year, everything())
```

4.2 Location Features

Column: geohash

Location-based features were derived to enable geospatial analysis and to help understand geographic patterns in property prices. The geohash column was decoded into latitude and longitude coordinates using the geohashTools package. These coordinates provide precise geographic locations of each property. Additionally, the distance from each property to a central location (e.g., Dublin city center) was calculated using the geosphere package, allowing for the analysis of the proximity of properties to key areas like city centers or amenities. The distance_to_DublinCity column indicates how far each property is from Dublin's city center in meters. These features are particularly useful for analyzing how location impacts property prices

Purpose:

By incorporating geographic data, properties can be better analyzed in relation to their location. Distance from key areas like city centers can be a strong factor influencing property prices, with properties closer to amenities and infrastructure often commanding higher prices. This analysis provides valuable insights into how geographic factors contribute to property valuation

Method:

- **Geohash Decoding:** The geohash column was decoded into latitude and longitude coordinates using the geohashTools package, enabling precise geospatial analysis of the properties' locations.
- **Distance Calculation:** Using the geosphere package, the distance from each property to Dublin city center was calculated. The distVincentySphere() function in the geosphere package computes the geodesic distance between two points on the Earth's surface, based on their latitude and longitude.
- **New Columns:** The latitude and longitude columns were created from the decoded geohash, and a new column, distance_to_DublinCity, was created to store the computed distance in meters.

This method decodes the geohash to latitude and longitude, calculates the distance from each property to Dublin city center, and creates new columns for latitude, longitude, and distance. It also includes handling missing values and visualizing the properties on a map for geospatial analysis.

```
##Locationfeature
#Install packages only if they are not already installed
if (!requireNamespace("geohashTools", quietly = TRUE)) {
  install.packages("geohashTools")
}
if (!requireNamespace("geosphere", quietly = TRUE)) {
  install.packages("geosphere")
}
if (!requireNamespace("sf", quietly = TRUE)) {
  install.packages("sf")
}

#Load the libraries
library(geohashTools)
library(geosphere)
library(sf)


#Assuming propertydataset1 has a column 'geohash' that stores geohash strings
all_properties_data <- all_properties_data %>%
  mutate(
    lat = geohashTools::gh_decode(geohash)$latitude,  # Decode geohash to latitude
    lon = geohashTools::gh_decode(geohash)$longitude # Decode geohash to longitude
  )

#City center coordinates (Dublin city center as an example)
city_lat <- 53.349805
city_lon <- -6.26031

#Calculate the distance (in meters) from each property to the city center
#In meters
all_properties_data <- all_properties_data %>%
```

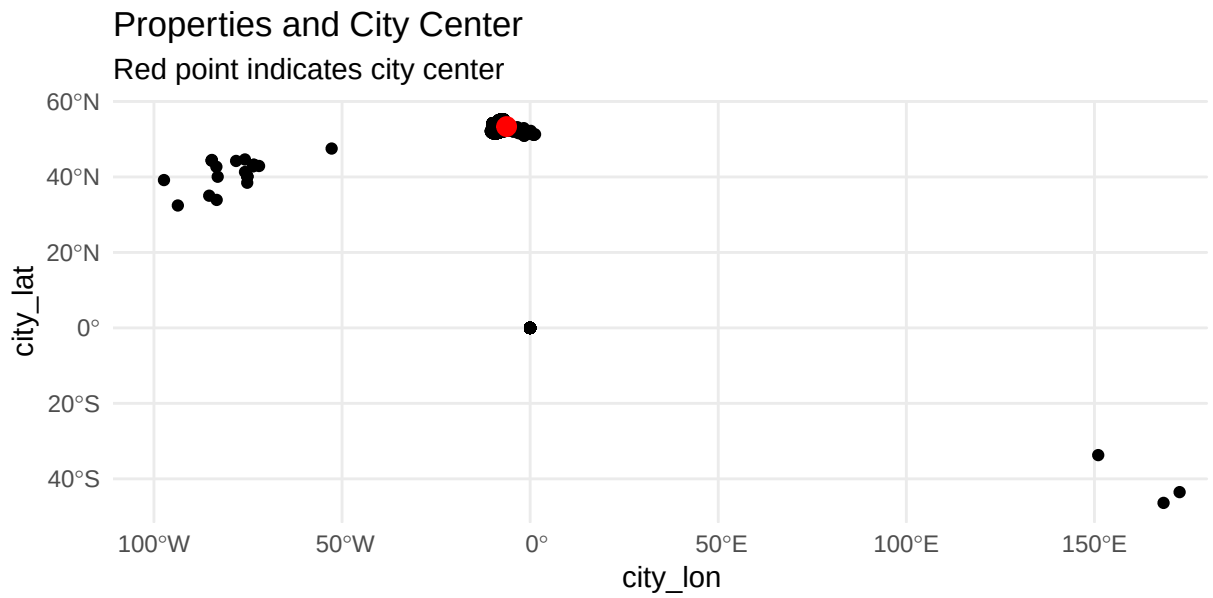
```
  mutate(
    distance_to_DublinCity = distVincentySphere(cbind(lon, lat), c(city_lon, city_lat))
  )

#Move columns
all_properties_data <- all_properties_data %>%
  select(geohash,lat, lon, distance_to_DublinCity, everything())

#Handle Missing Values in lat and lon:
#If some properties do not have valid geohashes, this will impute or exclude
all_properties_data <- all_properties_data %>%
  mutate(
    lat = ifelse(is.na(lat), 0, lat),  # Impute default value or handle appropriately
    lon = ifelse(is.na(lon), 0, lon)
  )

#Create a spatial dataframe from the latitude and longitude of the properties
property_sf <- st_as_sf(all_properties_data, coords = c("lon", "lat"), crs = 4326)

#Plot properties on a map
ggplot() +
  geom_sf(data = property_sf) +
  geom_point(aes(x = city_lon, y = city_lat), color = "red", size = 3) + #Plot city center
  theme_minimal() +
  labs(title = "Properties and City Center", subtitle = "Red point indicates city center")
```



4.3 Price Range Categorization

Column: price

This section categorizes the property prices into four distinct groups based on quantiles (percentiles) of the price distribution: Low, Medium, High, and Very High. This categorization provides a clearer understanding of the price distribution and allows for better segmentation of properties into different price groups

Purpose:

By categorizing properties into different price bands, we can better understand the distribution of property prices and segment the market based on affordability or luxury. This segmentation is useful for identifying trends, targeting specific buyer groups, and conducting more refined market analysis

Method:

The cut() function is used to group property prices into the defined categories based on the quantiles of the price distribution. These quantiles are calculated from the 25th percentile (Q1), 50th percentile (Q2, which is also the median), and 75th percentile (Q3). Based on these quantiles, the properties are categorized as follows:

- **Low:** Properties priced below the 25th percentile
- **Medium:** Properties priced between the 25th and 50th percentiles
- **High:** Properties priced between the 50th and 75th percentiles
- **Very High:** Properties priced above the 75th percentile

Steps:

1. **Calculate Quantiles:** Calculate the 25th, 50th, and 75th percentiles of the property price distribution using the quantile() function
2. **Categorize Prices:** The cut() function is used to group the prices into the defined categories based on the calculated quantiles. A new column, price_category, is created to store the price group for each property. This helps in categorizing properties into relevant price bands for further analysis
3. **Reorder Columns:** After categorizing the prices, the price_category column is moved to appear next to the price column for better organization and easier reference. This step enhances the readability of the dataset

The price_category column is now available for deeper insights and can be used for segmenting properties when conducting further analysis or building predictive models. This categorization helps identify how properties are distributed across different price ranges

```r
##Price Range
#Calculate quantiles
Q1 <- quantile(all_properties_data$price, 0.25, na.rm = TRUE)
Q2 <- quantile(all_properties_data$price, 0.50, na.rm = TRUE)  # This is the median
Q3 <- quantile(all_properties_data$price, 0.75, na.rm = TRUE)

#Categorize prices into low, medium, high based on the quantiles
all_properties_data$price_category <- cut(
  all_properties_data$price,
  breaks = c(-Inf, Q1, Q2, Q3, Inf),  # Define breaks for low, medium, and high
  labels = c("Low", "Medium", "High", "Very High"),  # 4 labels for 4 intervals
  right = FALSE
)
```

```r
#Check the first few rows to see the new column
invisible(head(all_properties_data))

#Find the position of the 'price' column
price_index <- which(names(all_properties_data) == "price")

#Reorder the columns to place 'price_category' right after 'price'
all_properties_data <- all_properties_data %>%
  select(1:price_index, price_category, (price_index + 1):ncol(all_properties_data))

#Check the first few rows to see the result
invisible(head(all_properties_data))
```

4.4 Price Trend Based on Year

Column: price_trend

The price_trend column helps to categorize the price movement of properties over time by comparing the price of a property in the current year with its price in the previous year. This classification enables us to classify the trend as increasing, decreasing, or stable, providing insights into whether property prices are trending upwards, downwards, or remaining constant over time

Purpose:

By examining price trends, we can determine how property values change year-over-year, revealing insights into market dynamics and helping to predict future trends. This categorization also highlights the impact of external factors (like economic shifts) on property prices

Method:

Data Comparison: Each property's price is compared with the price from the previous year to determine the trend. If the current price is greater than the previous year's price, the trend is classified as "Increasing". If it is lower, the trend is "Decreasing". If the price has not changed, the trend is "Stable"

lag() Function: The lag() function retrieves the price of the property from the previous sale, allowing for year-to-year comparison

Trend Categories: The price_trend column is classified into three categories:

- **Increasing:** Current price is higher than the price from the previous year
- **Decreasing:** Current price is lower than the previous year's price
- **Stable:** Price remains the same as the previous year

```r
##PriceTrend
#Create 'price_trend' column based on year-to-year price change
all_properties_data$price_trend <- ifelse(
  all_properties_data$price > lag(all_properties_data$price),
  "Increasing",
  ifelse(
    all_properties_data$price < lag(all_properties_data$price),
    "Decreasing",
    "Stable"  # If price is the same, it's stable
  )
)
#Find the position of the 'price_category' column
```

```r
price_category_index <- which(names(all_properties_data) == "price_category")

#Reorder the columns to place 'price_trend' right after 'price_category'
all_properties_data <- all_properties_data %>%
  select(1:price_index, price_category, price_trend,
         (price_category_index + 1):ncol(all_properties_data))
```

4.5 Market Conditions

Column: market_condition

The market_condition column helps classify the market conditions based on the sale date. By using the month of sale, we can categorize the market as either more favorable for buyers or sellers. For instance:

- **Buyer's Market** may be linked to months where demand is typically lower (e.g., winter months)
- **Seller's Market** could correspond to months of higher demand (e.g., spring and summer months)

Purpose:

This categorization helps explain price fluctuations due to market dynamics and provides context to why certain properties may have sold at higher or lower prices depending on the market conditions at the time

Method:

The case_when() function is used to categorize market conditions based on the month of the sale. The sale month is extracted from the sale_date column, and properties are classified into either a buyer's or seller's market, based on typical seasonal trends in demand

```r
##MarketCondition
#Create a market_condition column based on the month of sale
all_properties_data$market_condition <- ifelse(
  format(all_properties_data$sale_date, "%m") %in%
    c("12", "01", "02"), "Buyer's Market",   # Winter months
  ifelse(format(all_properties_data$sale_date, "%m") %in%
           c("06", "07", "08"), "Seller's Market","Neutral Market")
  #Summer months for Seller's and Default to Neutral for other months
)

#Check the first few rows to see the new column
invisible(head(all_properties_data))

#Find the position of the 'sale_year' column
sale_year_index <- which(names(all_properties_data) == "sale_year")

#Reorder the columns to place 'market_condition' right after 'sale_year'
all_properties_data <- all_properties_data %>%
  select(1:sale_year_index, market_condition,
         (sale_year_index + 1):ncol(all_properties_data))
```

4.6 Seasonal Factors: Month or Quarter of Sale

Column: seasonal_factor, sale_quarter

The month or quarter in which a property is sold can greatly influence its price due to seasonal demand fluctuations. During peak seasons (e.g., spring or summer), property prices may be higher due to increased demand. Conversely, in the colder months, there might be lower demand, leading to lower prices

Purpose: Including seasonal factors as variables can enhance price predictions by accounting for seasonal variations in property prices. This provides better insights into how the timing of a sale may influence the final price of a property

Seasonal Factor: The seasonal_factor column categorizes sales into seasons based on the month of the sale:

- **Spring:** March, April, May
- **Summer:** June, July, August
- **Autumn:** September, October, November
- **Winter:** December, January, February

Sale Quarter: The sale_quarter column categorizes sales by the quarter of the year:

- **Q1:** January, February, March
- **Q2:** April, May, June
- **Q3:** July, August, September
- **Q4:** October, November, December

Method: The case_when() function is used to classify properties into seasonal categories based on the month of the sale. The month is extracted from the sale_date column, and based on this, the properties are assigned to the correct season or quarter

```
##SeasonalMarket
#Add 'seasonal_factor' column based on the sale month
#This categorizes sales based on the month of the sale, such as spring, summer, autumn, or winter

all_properties_data$seasonal_factor <- case_when(
  format(all_properties_data$sale_date, "%m") %in%
    c("03", "04", "05") ~ "Spring",    # March, April, May
  format(all_properties_data$sale_date, "%m") %in%
    c("06", "07", "08") ~ "Summer",    # June, July, August
  format(all_properties_data$sale_date, "%m") %in%
    c("09", "10", "11") ~ "Autumn",    # September, October, November
  format(all_properties_data$sale_date, "%m") %in%
    c("12", "01", "02") ~ "Winter",    # December, January, February
  TRUE ~ "Unknown"
)

#Add 'sale_quarter' column to categorize sales by quarter
#This groups sales into four quarters of the year based on the sale date

all_properties_data$sale_quarter <- case_when(
  format(all_properties_data$sale_date, "%m") %in%
    c("01", "02", "03") ~ "Q1",        # January, February, March
  format(all_properties_data$sale_date, "%m") %in%
    c("04", "05", "06") ~ "Q2",        # April, May, June
  format(all_properties_data$sale_date, "%m") %in%
    c("07", "08", "09") ~ "Q3",        # July, August, September
  format(all_properties_data$sale_date, "%m") %in%
```

```
    c("10", "11", "12") ~ "Q4",        # October, November, December
  TRUE ~ "Unknown"
)
#Find the index of 'sale_date'
sale_date_index <- which(names(all_properties_data) == "sale_date")

#Reorder the columns to place 'seasonal_factor' and 'sale_quarter' after 'sale_date'
all_properties_data <- all_properties_data %>%
  select(1:sale_date_index, seasonal_factor, sale_quarter,
         (sale_date_index + 1):ncol(all_properties_data))

#Check the first few rows to see the newly added columns
invisible(head(all_properties_data))
```

4.7 Property Type Classification

Column: property_description

To enhance the dataset and provide more explanatory variables for modeling, the `property_description` column has been classified into broader property types. This classification is based on specific keywords or patterns within the property description, allowing for a better understanding of how different property types may influence property prices

Purpose:

By classifying properties into more general categories, we can better analyze trends and make comparisons between property types, allowing for improved modeling. This can help identify which types of properties are more likely to have higher or lower prices and provide insights into market segmentation

Method:

Using the `case_when()` function in R, each property description is checked for certain keywords, and a corresponding property type is assigned. The property types created are as follows:

- **"Dwelling House/Apartment"**: Includes both new and second-hand dwelling houses and apartments
- **"Semi-Detached House"**: For properties labeled as semi-detached houses
- **"Terraced House"**: For terraced houses and end-of-terrace houses
- **"Apartment"**: Specifically for properties described as apartments
- **"Detached House"**: For detached houses
- **"House"**: For general houses (e.g., "House For Sale")
- **"Bungalow"**: For properties described as bungalows
- **"Duplex"**: For duplex properties
- **"Townhouse"**: For townhouse properties
- **"Site"**: For properties labeled as "Site For Sale"
- **"Other"**: For descriptions that don't match any of the above categories

By adding the property_type column, the dataset now includes more specific property classifications that help in analyzing the relationship between property type and price, improving model predictions and analysis

```
##PropertyType
#Group the property types
all_properties_data$property_type <- case_when(
  grepl("Dwelling House/Apartment", all_properties_data$property_description,
```

```
        ignore.case = TRUE) ~ "Dwelling House/Apartment",
  grepl("Semi-Detached House", all_properties_data$property_description,
        ignore.case = TRUE) ~ "Semi-Detached House",
  grepl("Terraced House|End Of Terrace House", all_properties_data$property_description,
        ignore.case = TRUE) ~ "Terraced House",
  grepl("Apartment For Sale", all_properties_data$property_description,
        ignore.case = TRUE) ~ "Apartment",
  grepl("Detached House", all_properties_data$property_description,
        ignore.case = TRUE) ~ "Detached House",
  grepl("House For Sale", all_properties_data$property_description,
        ignore.case = TRUE) ~ "House",
  grepl("Bungalow For Sale", all_properties_data$property_description,
        ignore.case = TRUE) ~ "Bungalow",
  grepl("Duplex For Sale", all_properties_data$property_description,
        ignore.case = TRUE) ~ "Duplex",
  grepl("Townhouse", all_properties_data$property_description,
        ignore.case = TRUE) ~ "Townhouse",
  grepl("Site For Sale", all_properties_data$property_description,
        ignore.case = TRUE) ~ "Site",
  TRUE ~ "Other"
)
#Find the index of 'property_description'
property_description_index <- which(names(all_properties_data) == "property_description")

#Reorder the columns to place 'property_type' right after 'property_description'
all_properties_data <- all_properties_data %>%
  select(1:property_description_index, property_type,
         (property_description_index + 1):ncol(all_properties_data))

#Check the first few rows to see the classification
invisible(head(all_properties_data))
```

5 Visualization

5.1 Price Distribution by Property Type

This density plot illustrates the distribution of property prices for various property types, such as detached houses, apartments, and more. The chart reveals the shape of the price distribution for each property type, highlighting overlaps or distinctions among them. It provides insights into which property types typically have higher or lower prices and the range of prices within each category.

5.1.1 Explanation:

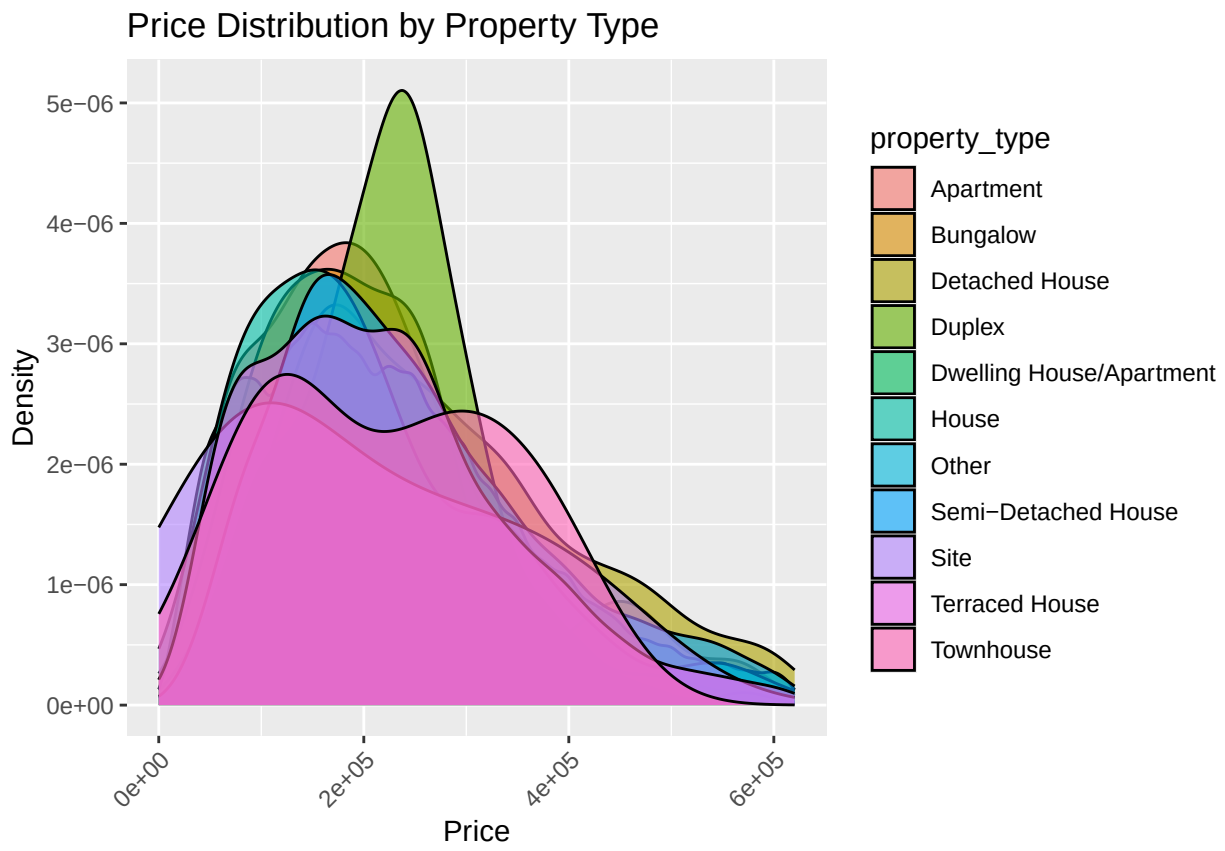
Purpose: The density plot is useful for comparing the spread and central tendencies of property prices across different property types. It shows whether certain property types cluster around specific price ranges or have wider price variability.

Interpretation:

- The geom_density() function is used to create the density plot for visualizing the distribution of prices
- aes(color = property_type) assigns different colors to each property type for clear differentiation
- theme_minimal() is applied to give the plot a simple and uncluttered look

```
#Price distribution by Property Type using Density Plot

ggplot(all_properties_data, aes(x = price, fill = property_type)) +
  geom_density(alpha = 0.6) +
  labs(title = "Price Distribution by Property Type", x = "Price", y = "Density") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



5.2 Price Trend By Time (By Year)

This line chart displays how the average property price has evolved over time on a yearly basis. By plotting the mean price for each year, the chart highlights trends, such as whether property prices have been rising, falling, or remaining steady over the years. The visualization offers a clear picture of the temporal dynamics of property prices.

5.2.1 Explanation:

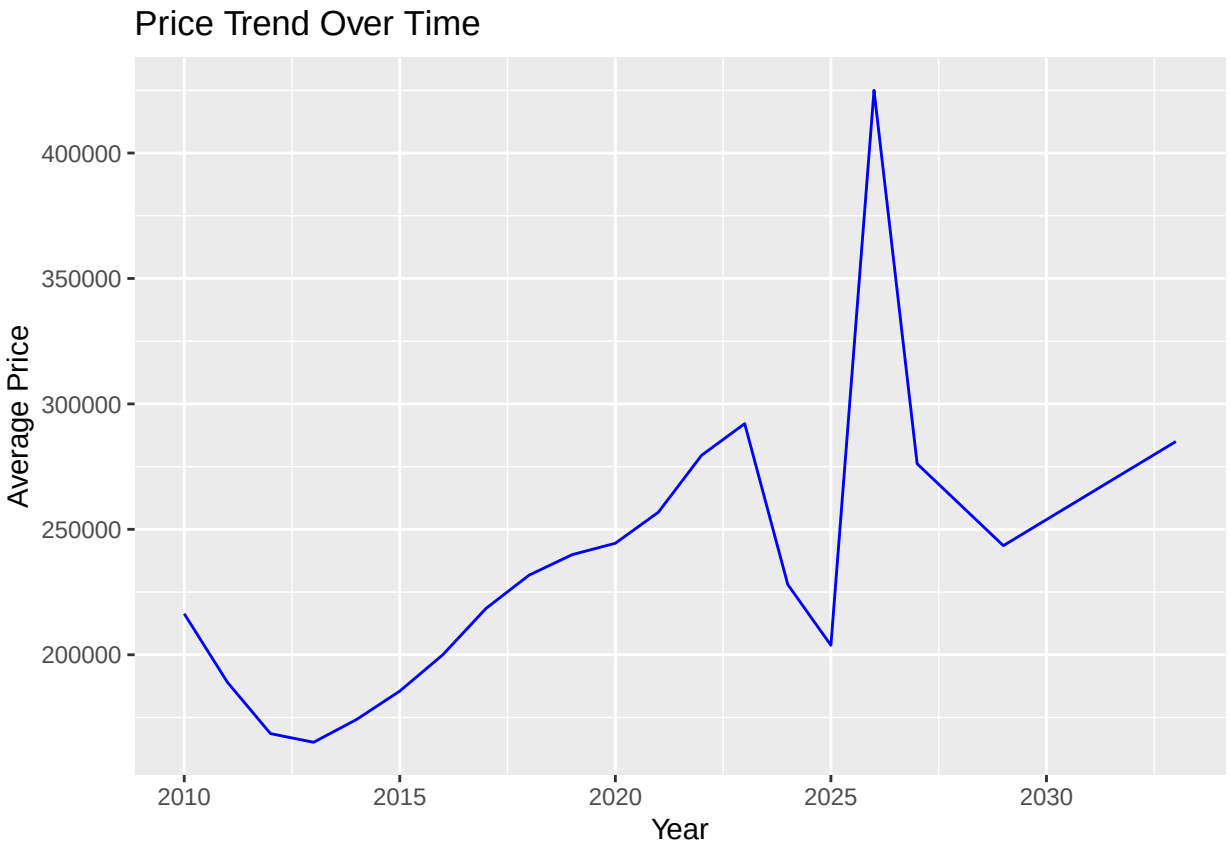
Purpose: The line chart effectively tracks changes in the average property price over time, making it easy to observe long-term trends in the market.

Interpretation:

- stat_summary() is used to calculate and display the average (fun = "mean") property price for each year as a line plot (geom = "line")
- The line is styled with the color = "blue" argument, which can be customized for aesthetic preferences
- The `theme_minimal()` provides a clean, simple theme to make the chart more readable

```
#Price Trend Over Time (by Year)

ggplot(all_properties_data, aes(x = sale_year, y = price)) +
  stat_summary(fun = "mean", geom = "line", color = "blue") +
  labs(title = "Price Trend Over Time", x = "Year", y = "Average Price")
```



5.3 Market Condition vs. Price

This box plot illustrates property prices under different market conditions, such as buyer's market or seller's market. Each box represents the price distribution for a specific market condition, showcasing the variability and central tendency of prices. The visualization helps in identifying trends, such as whether properties are generally priced higher in a seller's market compared to a buyer's market. Wider or taller boxes indicate more variability in prices, while shorter boxes reflect more consistent pricing within the market condition.

5.3.1 Explanation:

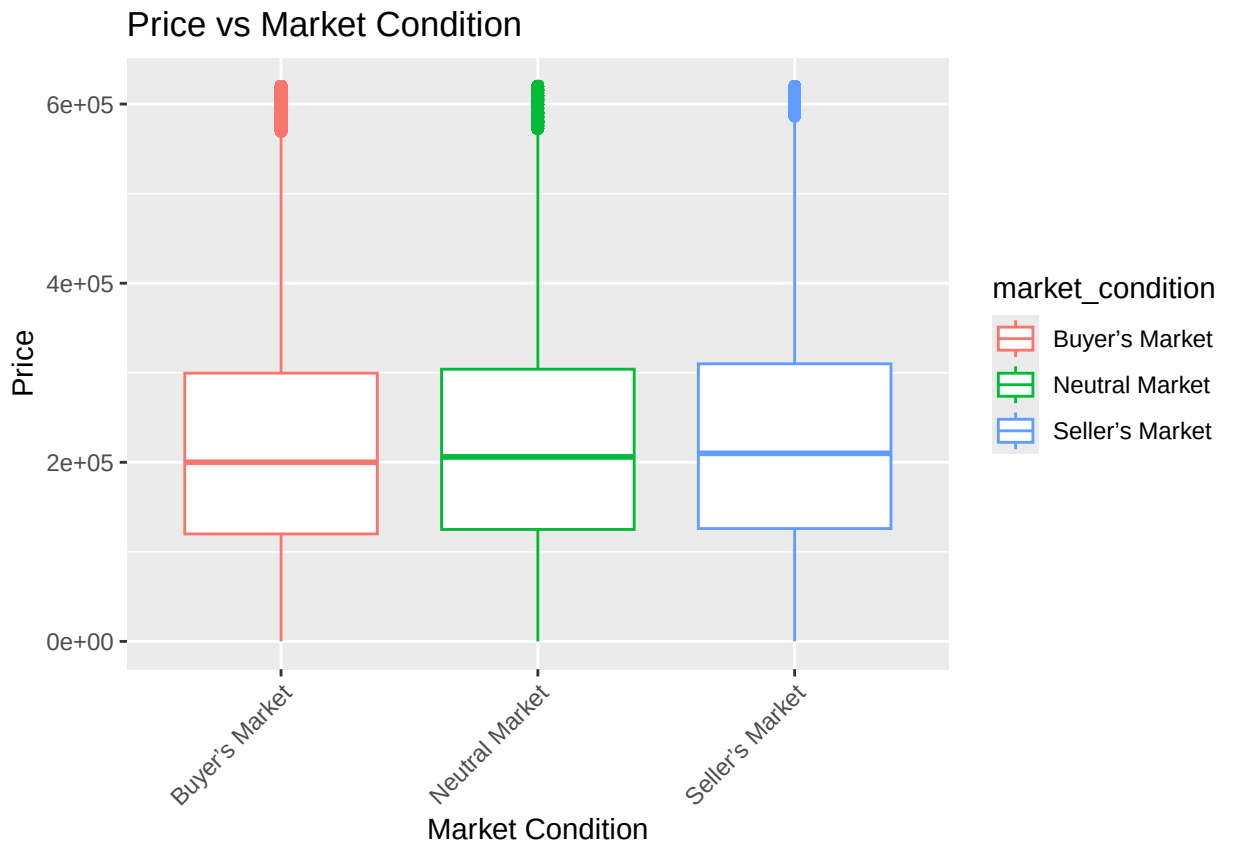
Purpose: The box plot visually compares the range and median property prices across various market conditions. It highlights the spread of prices, making it easier to identify differences between market conditions.

Interpretation:

- The geom_boxplot() function is used to create the box plot.
- The aes(color = market_condition) argument applies distinct colors to each market condition, aiding visual distinction.
- theme_minimal() provides a clean and professional layout.
- theme(axis.text.x = element_text(angle = 45, hjust = 1)) adjusts the orientation of x-axis labels to improve readability, especially for longer labels.

```
#Market Condition vs Price
```

```r
ggplot(all_properties_data, aes(x = market_condition, y = price)) +
  geom_boxplot(aes(color = market_condition)) +
  labs(title = "Price vs Market Condition", x = "Market Condition", y = "Price") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



5.4 Price Distribution by County

This bar plot displays the average property price for each county. Each bar represents a county, with the height reflecting the average price of properties sold in that region. The visualization identifies areas with higher or lower average prices, offering insights into regional price variations in the property market. Taller bars indicate counties with higher property prices, while shorter bars represent more affordable regions. This visualization highlights geographical differences in property values and regional market trends.

5.4.1 Explanation:

Purpose: The bar plot is designed to compare the average property price across counties. It highlights variations in property values, making it easier to identify regions with higher or lower average prices.

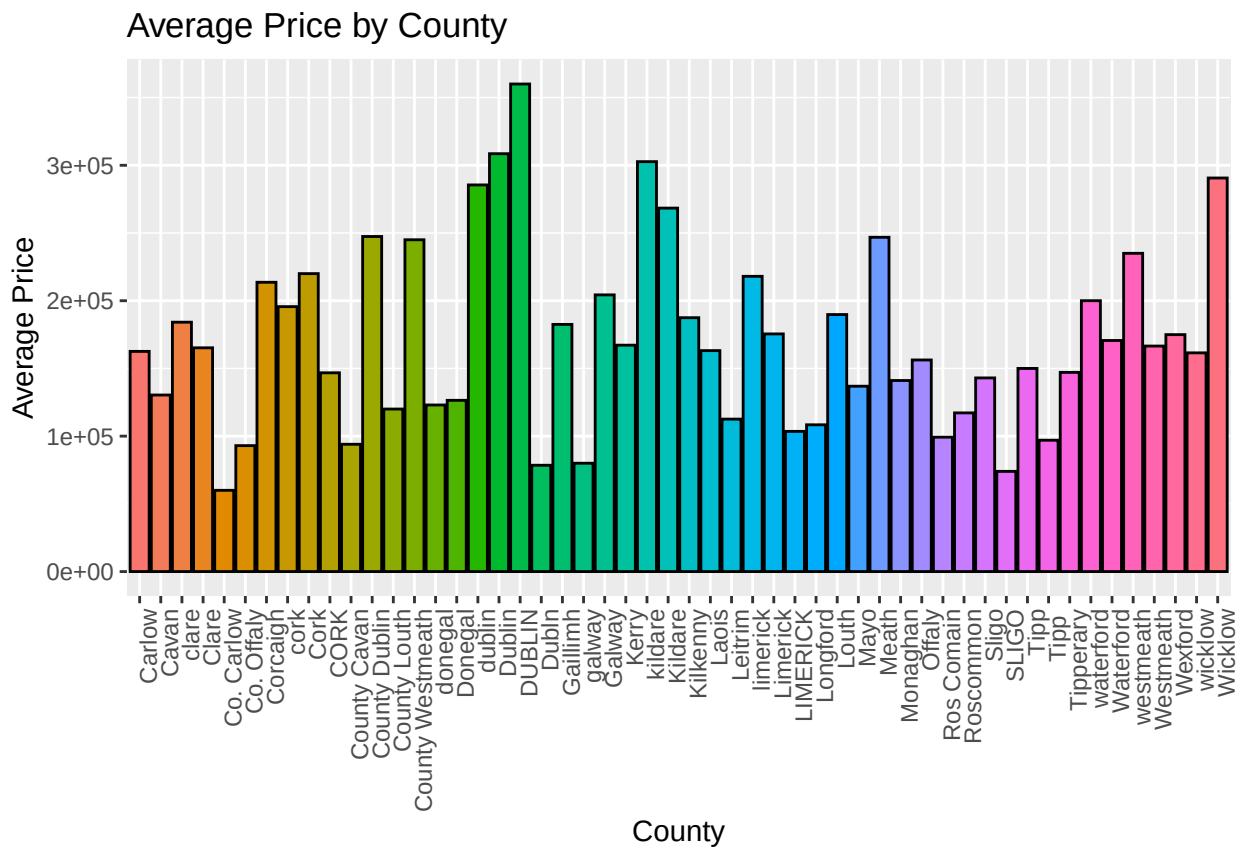
Interpretation:

- The stat_summary(fun = "mean", geom = "bar", color = "black") is used to calculate and display the average property price as a bar for each county
- theme_minimal() ensures a clean and professional layout

- theme(axis.text.x = element_text(angle = 90, hjust = 1)) rotates the county names on the x-axis for improved readability.
- scale_fill_manual(values = scales::hue_pal()(length(unique(all_properties_data$county)))) assigns distinct colors to counties, enhancing visual differentiation

```
#Price vs County using a Bar Plot (Average Price)

ggplot(all_properties_data, aes(x = county, y = price, fill = county)) +
  stat_summary(fun = "mean", geom = "bar", color = "black") +
  labs(title = "Average Price by County", x = "County", y = "Average Price") +
  theme(axis.text.x = element_text(angle = 90, hjust = 1),
        legend.position = "none") +
  scale_fill_manual(values = scales::hue_pal()
                    (length(unique(all_properties_data$county))))
```



5.5 Distribution of Properties Sold by County

This visualization uses a pie chart to display the proportion of properties sold across different counties. Each slice of the pie represents a county, with the size of the slice indicating the relative frequency of property sales in that county. The chart helps to identify which counties have a higher volume of property sales compared to others. For example, larger slices suggest counties with more sales activity, while smaller slices indicate fewer transactions. This visualization provides insights into regional market dynamics and can help to highlight areas of high or low demand in the property market. The chart is accompanied by a legend that labels each county.

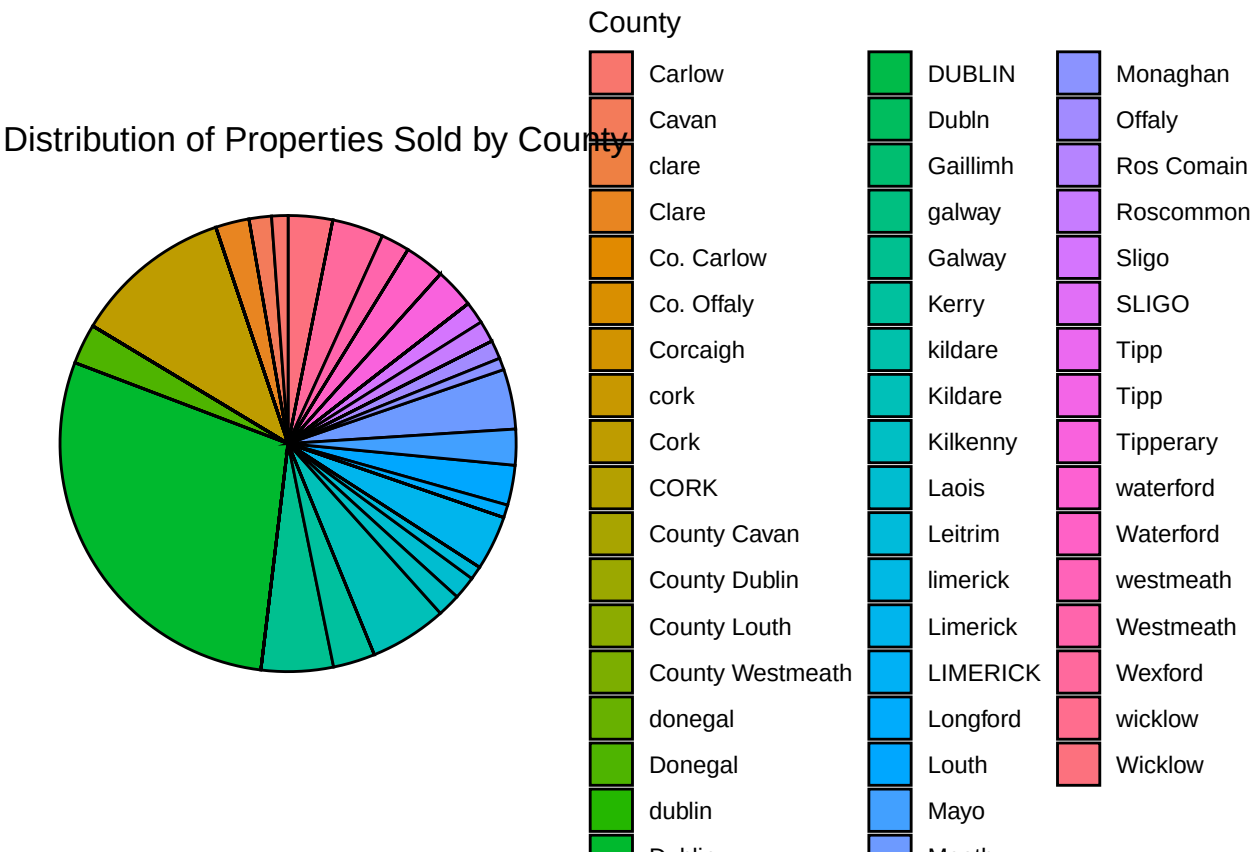
5.5.1 Explanation:

Purpose: The pie chart allows us to visually compare the sales activity in different counties. By looking at the size of the slices, we can easily identify which counties are seeing more or fewer property transactions. This can help understand regional trends and market demand, potentially guiding decisions related to investment, pricing, and marketing strategies.

Interpretation: Larger slices indicate counties with a higher volume of property sales, whereas smaller slices represent counties with fewer sales. This type of visualization helps in identifying areas with higher market activity, providing insights into regional demand for properties.

```
#Calculate the count of properties by county
county_distribution <- all_properties_data %>%
  group_by(county) %>%
  summarize(count = n()) %>%
  arrange(desc(count))

#Create a pie chart for county distribution
ggplot(county_distribution, aes(x = "", y = count, fill = county)) +
  geom_bar(stat = "identity", width = 1, color = "black") +
  coord_polar("y", start = 0) +
  labs(
    title = "Distribution of Properties Sold by County",
    fill = "County"
  ) +
  theme_void() +
  scale_fill_manual(values = scales::hue_pal()(length(unique(county_distribution$county))))
```



5.6 Heatmap: Average Price by County and Price Category

This heatmap provides a visualization of the average property price in different counties, categorized by price range (low, medium, high, very high). The counties are represented along the x-axis, while the price categories are shown along the y-axis. The color intensity in each cell indicates the average price, with darker colors representing higher average prices.

5.6.1 Explanation:

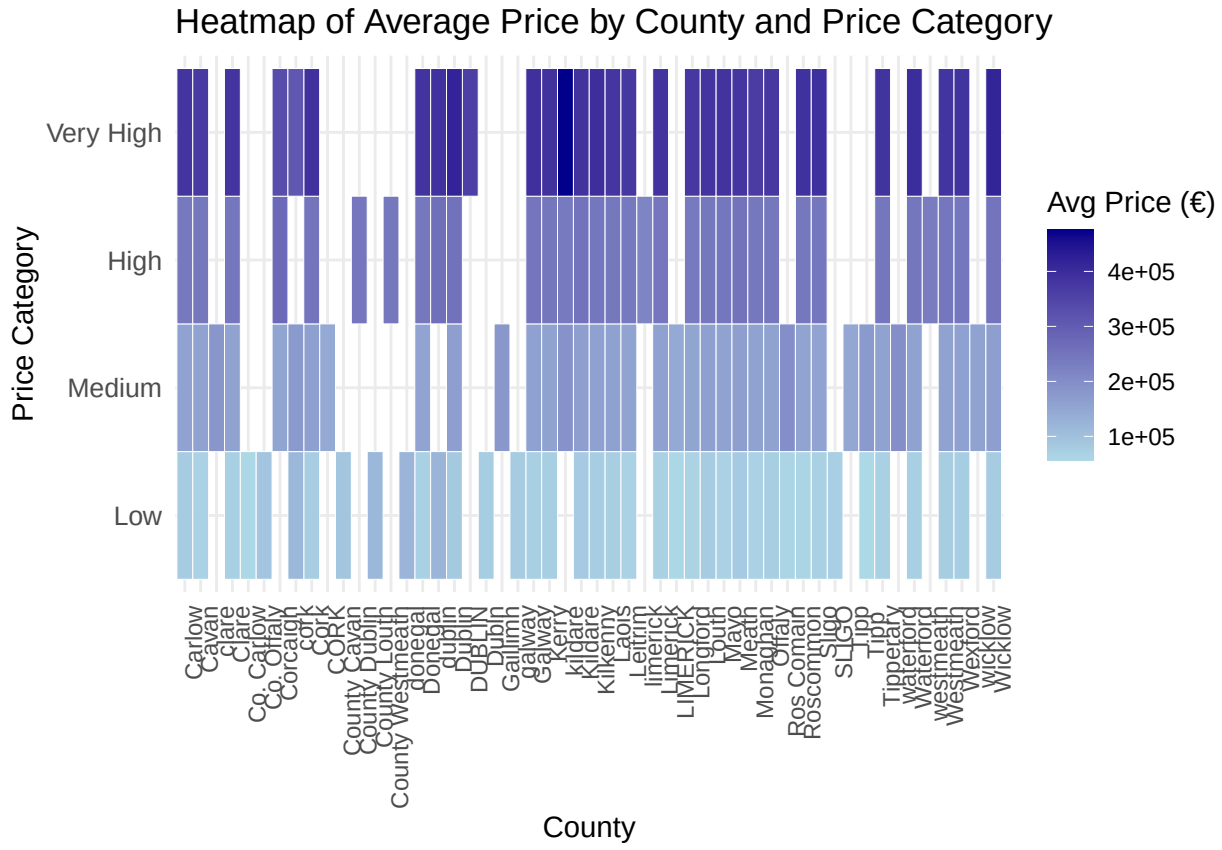
Purpose: This heatmap helps to identify regional variations in property prices, segmented by price categories. By examining the color intensity, it is easy to determine where the most expensive properties are located and how property prices are distributed across different counties.

Interpretation: Darker shades indicate higher average prices in a particular county and price category, while lighter shades suggest lower average prices. This visualization is useful for understanding which counties have higher property values and whether those higher prices are concentrated in specific price categories (e.g., "Very High" or "High").

```
#Calculate average price for each county and price category
heatmap_data <- all_properties_data %>%
  group_by(county, price_category) %>%
  summarise(avg_price = mean(price, na.rm = TRUE)) %>%
  ungroup()

#Create a heatmap

ggplot(heatmap_data, aes(x = county, y = price_category, fill = avg_price)) +
  geom_tile(color = "white") + # Add borders between tiles
  scale_fill_gradient(low = "lightblue", high = "darkblue", name = "Avg Price (€)") +
  labs(
    title = "Heatmap of Average Price by County and Price Category",
    x = "County",
    y = "Price Category"
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 90, hjust = 1),
    axis.text.y = element_text(size = 10)
  )
```


Heatmap of Average Price by County and Price Category
Price Category
Very High
High
Medium
Low
Avg Price (€)
4e+05
3e+05
2e+05
1e+05
Carlow
Cavan
Clare
Clare
Co. Carlow
Co. Offaly
Corcaigh
cork
CORK
cORK
County Cavan
County Dublin
County Louth
County Westmeath
donegal
Donegal
dublin
Dublin
DUBLIN
Dublin
Gaillimh
galway
Galway
Kerry
kildare
Kildare
Kilkenny
Laois
Leitrim
limerick
Limerick
LIMERICK
Longford
louth
Mayo
Meath
Monaghan
Offaly
Ros Comain
Roscommon
Sligo
SLIGO
Tipp
Tipp
Tipperary
waterford
Waterford
Westmeath
Wexford
Wicklow
Wicklow
County

6 Conclusion

After preparing and analyzing the property data, several important insights and recommendations have emerged, which will be valuable in building predictive models for property prices in Ireland. Below is a summary of the key findings and actionable considerations for the analysis:

6.1 Explanatory Variable

The dataset includes a wide range of variables such as property location, size, type, and market conditions. To enhance the models, additional features like seasonal trends, price categories, and property types were introduced. These explanatory variables provide a comprehensive view of property pricing dynamics, improving the accuracy of predictive models.

6.2 Patterns in Price:

- **Location Impact:** Properties near Dublin tend to have higher prices, likely due to increased urban demand.
- **Seasonal Trends:** There is a slight increase in property prices during the summer months, indicating a seasonal effect on pricing.
- **Market Conditions:** Prices vary considerably based on market conditions. For instance, in a "Seller's Market," properties tend to be more expensive compared to a "Buyer's Market."

6.3 Regional Variations

- **County-Level Analysis:** Significant regional price differences were observed, with counties like Dublin and Cork commanding higher prices compared to more rural areas.
- **Price Categories:** Price categories (low, medium, high, very high) are unevenly distributed across counties, which provides insights into affordability and premium property markets.

6.4 Property Type Impact:

- **Higher Prices for Detached Houses:** Detached and semi-detached houses generally fetch higher average prices compared to apartments or terraced houses.
- **Important for Segmentation:** Property type plays a crucial role in segmentation, significantly influencing property pricing. ## Modeling Recommendations:
- **Key Predictors:** To build an effective model, focus on predictors such as property type, location (latitude, longitude, county), property size, and market conditions. These factors show strong relationships with property prices.
- **Interaction Effects:** Explore interactions between variables such as county and property type, or market conditions and property size. This will help capture more nuanced relationships and improve model performance.
- **Outlier Management:** The dataset has been cleaned of outliers, but it's important to continue checking for any extreme values that might impact model stability and accuracy.

6.5 Visualizations

A range of visualizations, including scatter plots, box plots, heatmaps, and pie charts, were used to uncover trends and relationships within the data. These visual aids will be helpful for exploring data patterns further and generating hypotheses for the predictive model.

With this enriched and cleaned dataset, there is now a solid foundation to build predictive models for property prices in Ireland. These models will not only aid in identifying potential investment opportunities but also provide a deeper understanding of the factors driving property prices in different regions and property types.